Decision Trees & Random Forests

Lecture 5 — Machine Learning

Chen Hajaj

Agenda

1. Motivation — Why Decision Trees?
2. How a Tree Thinks — CART Algorithm
3. Splitting Criteria — Gini & Entropy
4. Regression Trees
5. Overfitting, Bias-Variance & Pruning
6. Random Forest — Bagging & Feature Randomness
7. Hyperparameter Tuning
8. Code Examples & Practice

1. Why Decision Trees?

Trees mimic human decision-making — sequences of yes/no questions

- ✅ Interpretable — rules can be printed and explained to anyone
- ✅ No feature scaling required
- ✅ Handles mixed types (numeric + categorical)
- ✅ Captures non-linear patterns automatically
- ✅ Built-in feature importance

- ❌ Single trees overfit easily (high variance)
- ❌ Cannot extrapolate beyond training range
- ❌ Sensitive to small data changes

The 20-Questions Intuition

Think of a decision tree as playing "20 Questions":

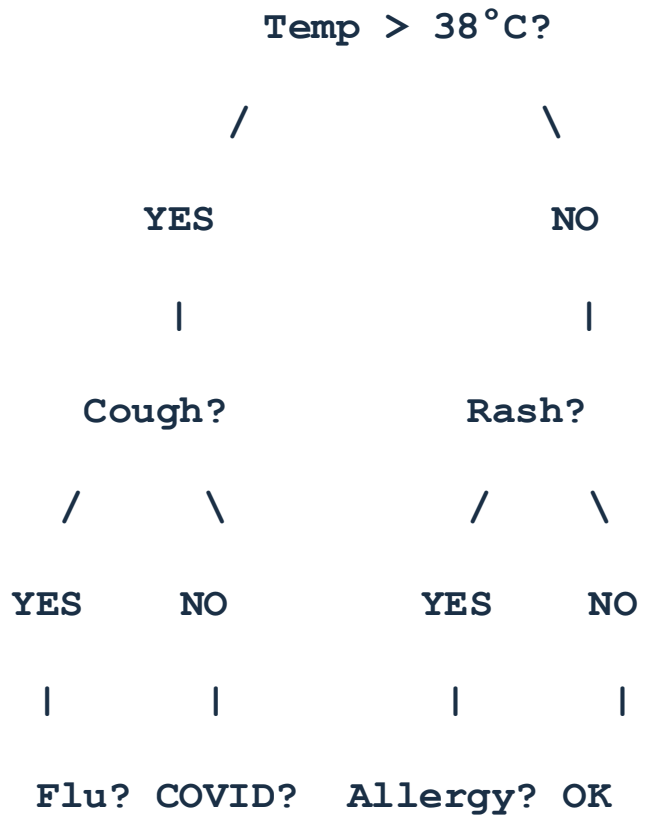
- Good question: "Is it alive?"

  Splits the space roughly in half

- Bad question: "Is it exactly 3.7cm?"

  Barely narrows anything down

Trees formalize this with Gini/Entropy to measure how much each question reduces uncertainty.



2. How a Tree Thinks — CART Algorithm

CART = Classification and Regression Trees (sklearn default)

At each node:

1. Try every feature × every threshold
2. Pick the split that maximally reduces impurity (Gini)
3. Recurse on left and right child nodes

Stop when any of:

- max_depth reached
- Node is pure (all same class)
- Too few samples (min_samples_split)

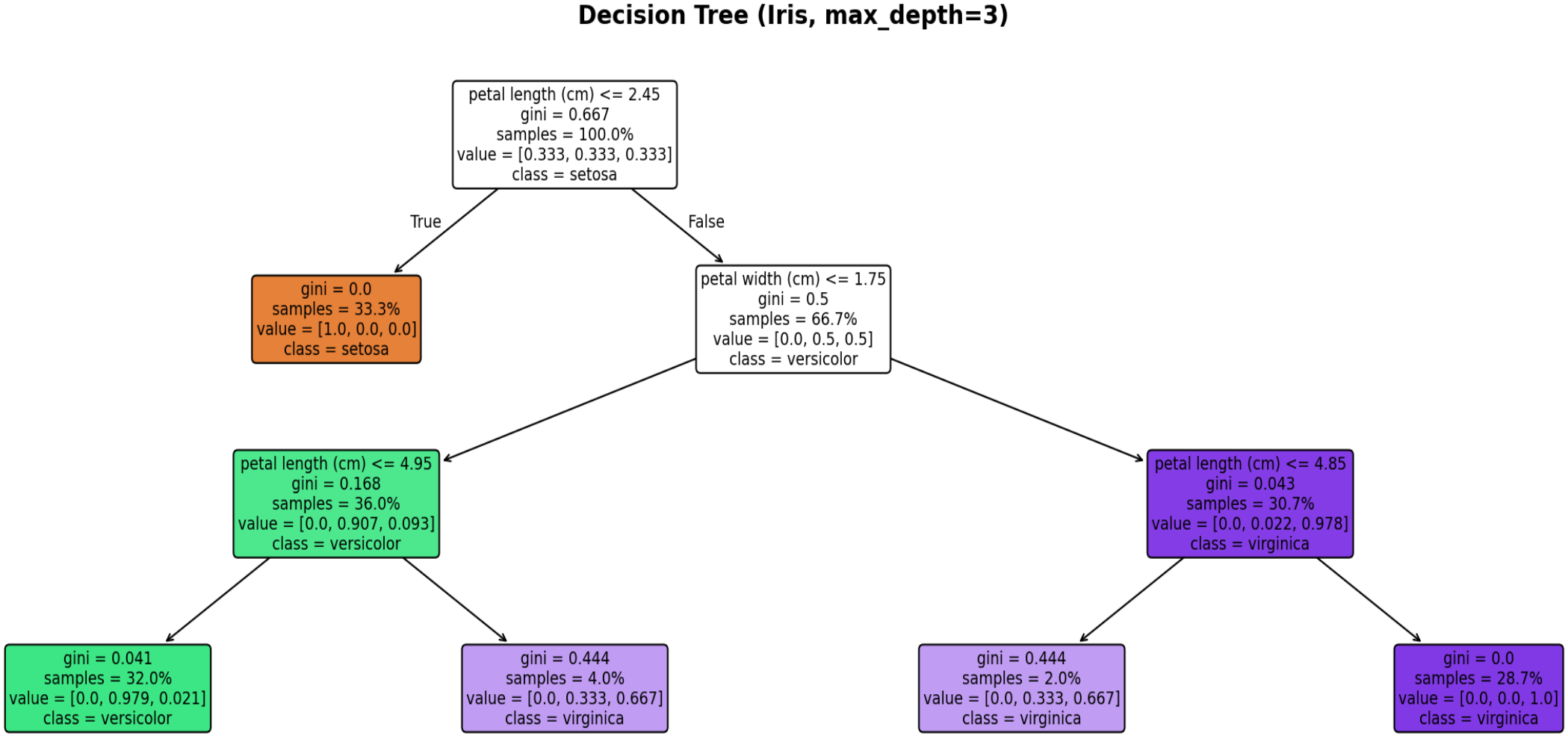
⚠ Greedy algorithm — locally optimal splits, not globally optimal tree

Choosing the Best Split — Toy Example

Tree Terminology

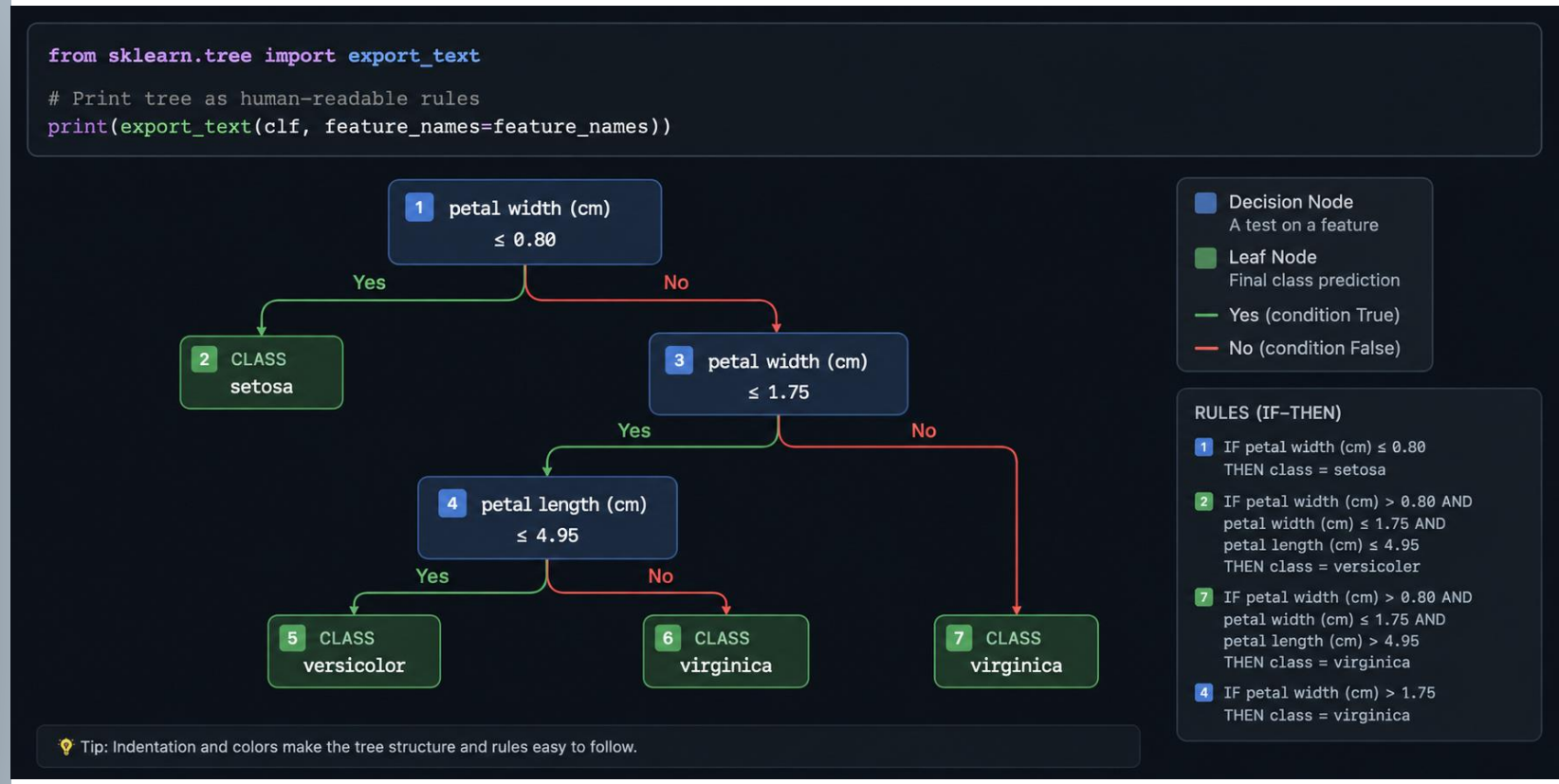
- Root Node - starting point; contains all samples
- Internal Node - split test on one feature; has children
- Leaf Node - terminal node; makes a prediction
- Depth - levels from root to deepest leaf
- Impurity - measure of class mixing (Gini, Entropy)
- Information Gain - impurity reduction from a split
- Pruning - removing branches to prevent overfitting
- OOB Samples - training points not seen by a given tree (RF)

Visualising a Trained Tree (Iris, max_depth=3)



Every Tree = A Set of If-Else Rules

```
from sklearn.tree import export_text

# Print tree as human-readable rules
print(export_text(clf, feature_names=feature_names))

|--- petal width (cm) <= 0.80
|   |--- class: setosa
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: versicolor
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: virginica
|   |--- petal width (cm) >  1.75
|   |   |--- class: virginica

# This can be deployed WITHOUT scikit-learn!
```



Decision Boundaries Are Axis-Aligned Rectangles

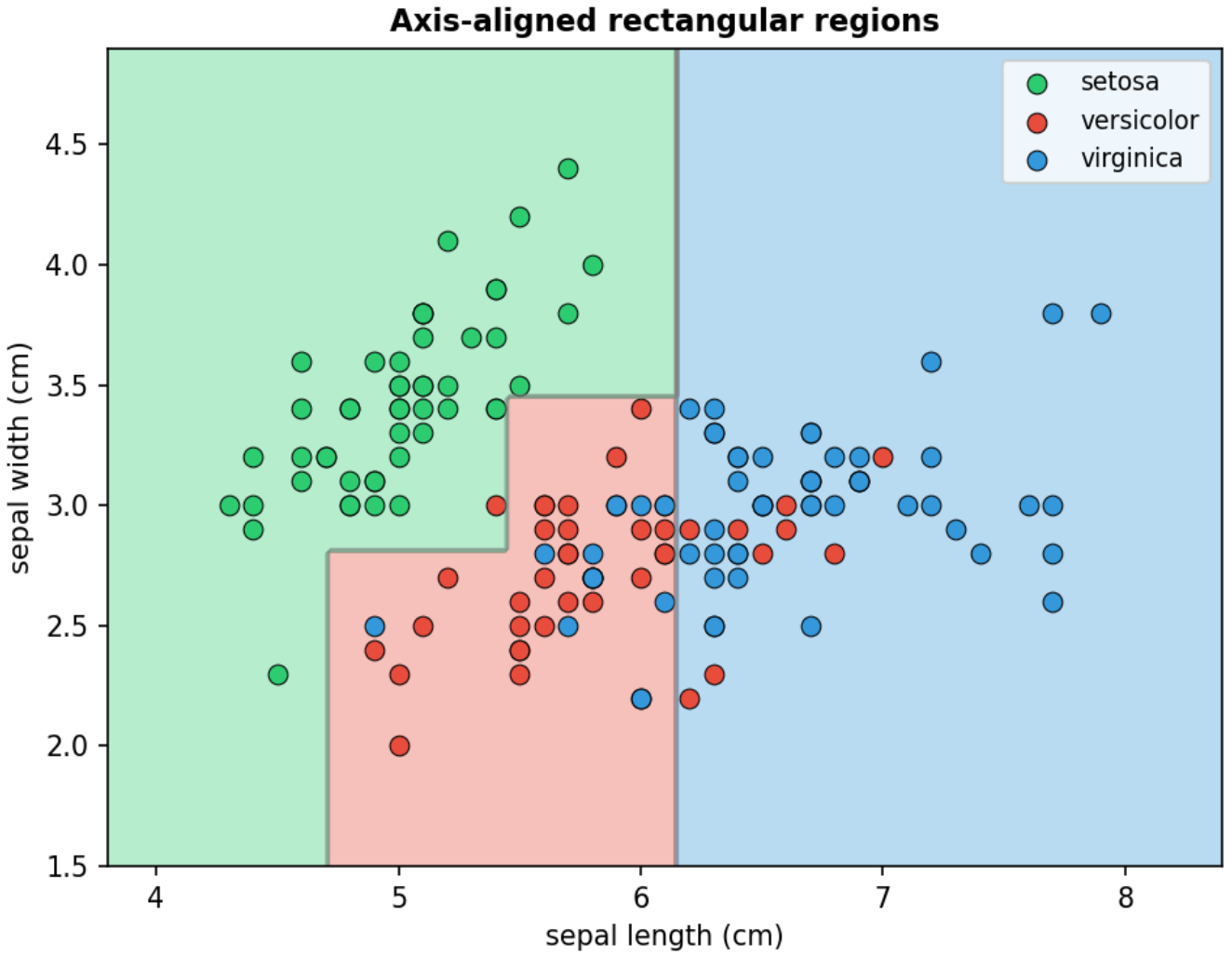
Key property of CART:

- All splits are perpendicular to a feature axis
- Creates rectangular decision regions

- Cannot learn diagonal boundaries directly
- Approximates them with many small rectangles

Implication:

- If true boundary is diagonal, tree needs many more splits than a linear model



Code: Training a Decision Tree

DECISION TREE CLASSIFIER — END TO END (Scikit-learn)

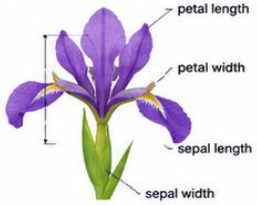
1. CODE WALKTHROUGH

```python
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

# Train
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)

# Evaluate
print(f"Train: {clf.score(X_train, y_train):.3f}  Test: {clf.score(X_test, y_test):.3f}")
print(f"Leaves: {clf.get_n_leaves()} | Depth: {clf.get_depth()}")

# Visualise & export
plot_tree(clf, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
print(export_text(clf, feature_names=list(iris.feature_names)))
```

2. WHAT THE CODE DOES

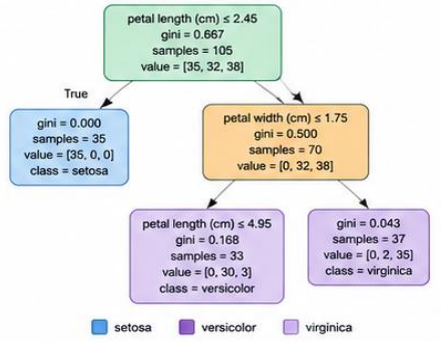
1	Import libraries	*DecisionTreeClassifier, plot_tree, export_text,* load_iris, train_test_split
2	Load Iris dataset	150 samples, 4 features, 3 classes (setosa, versicolor, virginica)
3	Train-test split	70% training, 30% testing (random_state=42 for reproducibility)
4	Train the model	Decision Tree (**max_depth=3**) Fit on training data
5	Evaluate model	Accuracy on train & test Number of leaves and depth of tree
6	Visualise & export	Plot the tree Print the tree structure as text

3. IRIS DATASET OVERVIEW



Samples	150
Features	4 numeric • sepal length (cm) • sepal width (cm) • petal length (cm) • petal width (cm)
Classes	3 • setosa (0) • versicolor (1) • virginica (2)

4. DECISION TREE (max_depth=3)



5. MODEL OUTPUT (Example)

```
Train: 1.000   Test: 0.978
Leaves: 4  |  Depth: 3
```

Interpretation:

- ✓ Model fits training data perfectly (depth=3).
- ✓ Test accuracy is ≈ **97.8%** (good generalization).
- ✓ Tree has **4** leaves and maximum depth **3**.

Per-class performance (Test set)

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	15
versicolor	0.95	1.00	0.97	15
virginica	1.00	0.93	0.97	15
accuracy			0.98	45

6. TEXT EXPORT (export_text output)

```
|--- petal length (cm) <= 2.45
|    |--- class:setosa
|--- petal length (cm) > 2.45
|    |--- petal width (cm) <= 1.75
|    |    |--- petal length (cm) <= 4.95
|    |    |    |--- class: versicolor
|    |    |--- petal length (cm) > 4.95
|    |    |    |--- class: virginica
|    |--- petal width (cm) > 1.75
|    |    |--- class: virginica
|
```

7. KEY PARAMETERS

Parameter	Value	Meaning
max_depth	3	Max depth of the tree (prevents overfitting)
random_state	42	Seed for reproducibility
test_size	0.3	30% data used for testing
criterion	gini (default)	Function to measure split quality
splitter	best (default)	Strategy to choose split at each node

Tip: Smaller max_depth → simpler tree (less overfitting)
Larger max_depth → more complex tree (may overfit)

8. GOOD PRACTICES

- ✓ Use train-test split to evaluate generalization.
- ✓ Tune max_depth, min_samples_split, etc.
- ✓ Visualize the tree to understand decisions.
- ✓ Check overfitting: train score >> test score ?
- ✓ Use cross-validation for robust evaluation.

Takeaway: Decision Trees are easy to understand and interpret. They can capture non-linear patterns but may overfit if the tree is too deep.

3. Splitting Criteria

Gini Impurity & Entropy

Impurity Measures — Formulas

Gini Impurity:

$G(p) = 2 \times p \times (1-p)$ [binary]

$G = 1 - \Sigma\, p_i^{\,2}$ [multi-class]

- Range: [0, 0.5]
- 0 = pure node | Max at p=0.5
- Default in sklearn (faster — no log)

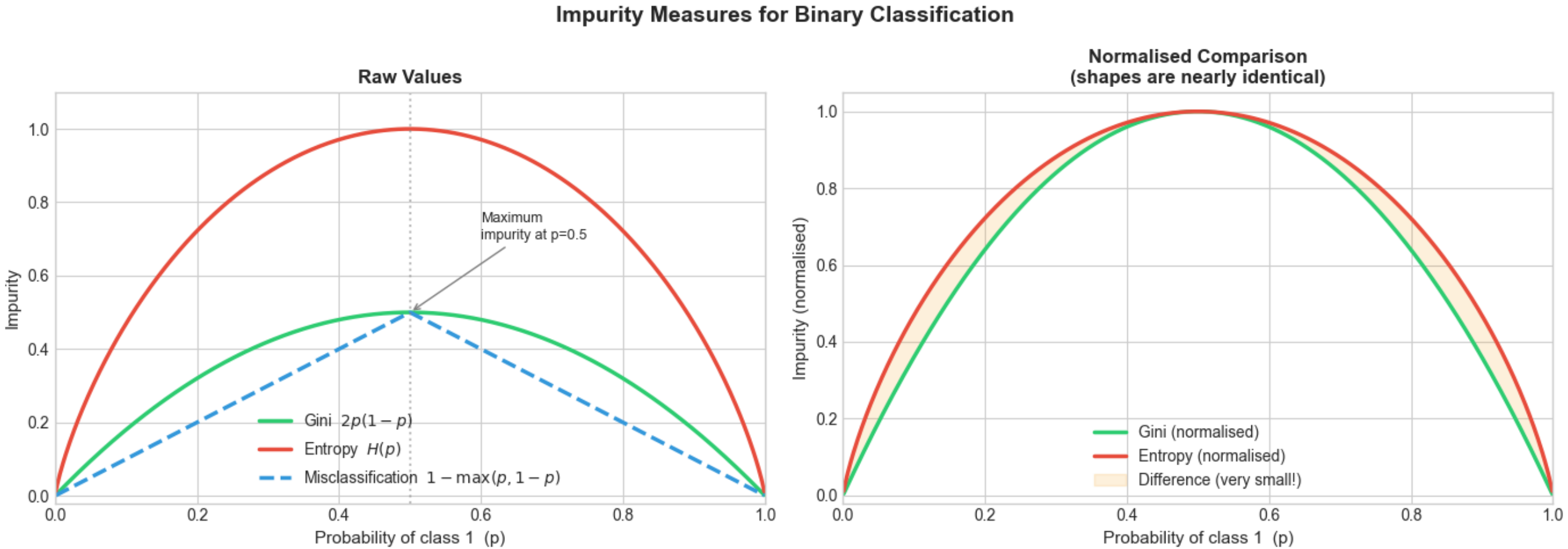
Entropy:

$H(X) = -p\,\backslash\log_2(p) - (1-p)\backslash\log_2(1-p)$ [binary]

$\mathrm{H(X)} = -\sum_{i=1}^{K} p_i \log_2(\mathrm{p_i})$ [mutl-class]

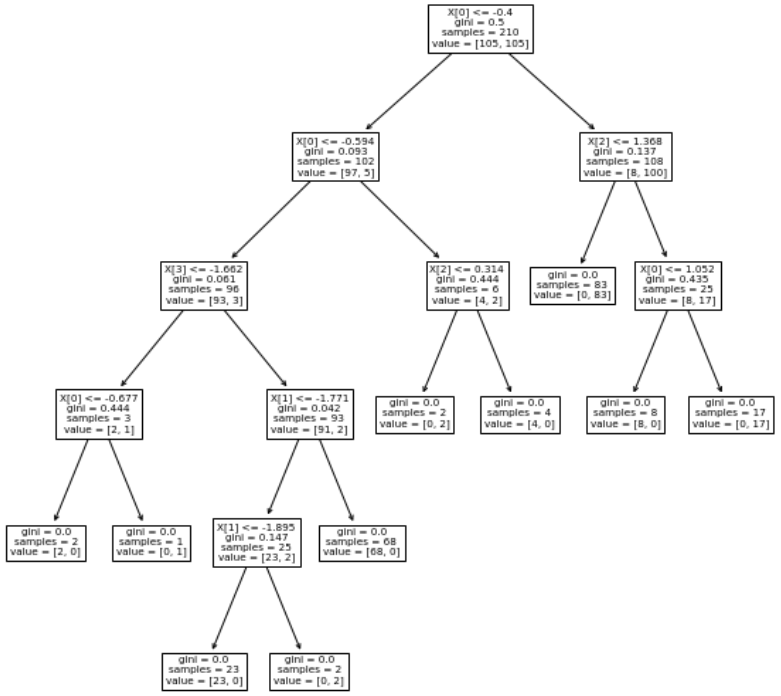
- Range: [0, 1]
- 0 = pure node | Max at p=0.5
- Slightly more sensitive near p=0.5
- Almost always same result as Gini

Gini vs Entropy vs Misclassification



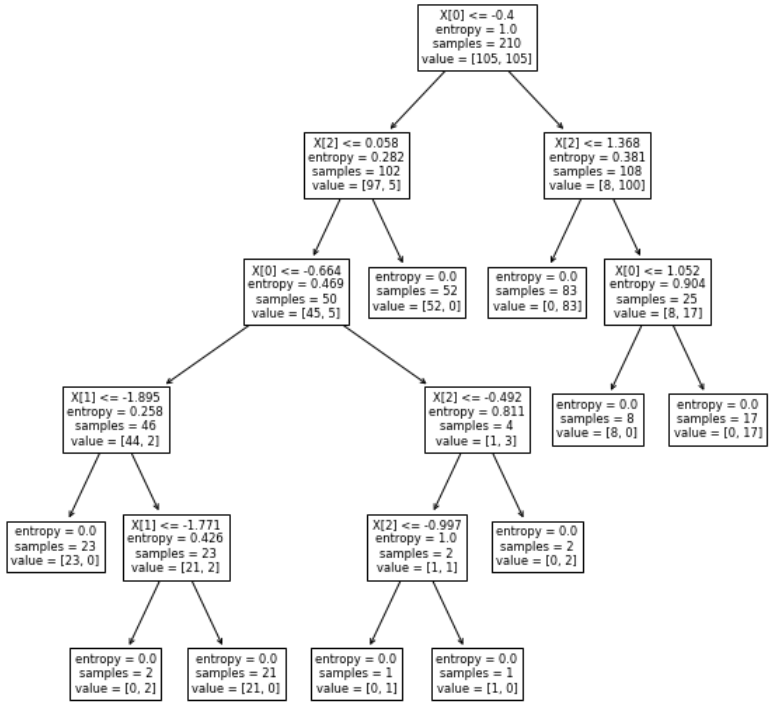
Gini vs Entropy

- Gini tree



VS

- Entropy tree



Faster | No log needed | Preferred default

More granular | Uses log | Similar splits in practice

Information Gain — Manual Calculation

Gini Impurity & Split Evaluation (Example)

DATASET

10 samples: 5 class 0, 5 class 1

Feature:

[1, 2, 3, 4, 5, 3, 6, 7, 8, 9]

Labels:

[0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

GINI FORMULA

$$Gini = 1 - \sum_{i=1}^{K} p_i^2$$

(For binary classes:

$Gini = 1 - (p_0{}^2 + p_1^2)$)

PARENT NODE (All data)

Class 0: 5 (50%)

Class 1: 5 (50%)

$Gini_{parent} = 1 - (0.5^2 + 0.5^2)$

$= 0.5000$

SUMMARY OF RESULTS

Split Rule	Weighted Gini	Gini Gain
feature ≤ 4	0.1667	0.3333
feature ≤ 5	0.1667	0.3333

☆ Both splits give the same Gini Gain (0.3333). This is a tie — pick either.

CANDIDATE SPLIT 1: feature ≤ 4

feature ≤ 4

Yes (≤ 4) | No (> 4)

LEFT CHILD

Labels: [0, 0, 0, 0]

Class 0: 4 (100%)

Class 1: 0 (0%)

$Gini_{left} = 1 - (1^2 + 0^2)$

$= 0.0000$

RIGHT CHILD

Labels: [0, 1, 1, 1, 1, 1]

Class 0: 1 (16.67%)

Class 1: 5 (83.33%)

$Gini_{right} = 1 - (0.83^2 + 0.17^2)$

$= 0.2778$

Weighted Gini = (4/10) × 0.0000 + (6/10) × 0.2778 = 0.1667

Gini Gain = $Gini_{parent}$ − Weighted Gini = 0.5000 − 0.1667 = **0.3333**

CANDIDATE SPLIT 2: feature ≤ 5

feature ≤ 5

Yes (≤ 5) | No (> 5)

LEFT CHILD

Labels: [0, 0, 0, 0, 0, 1]

Class 0: 5 (83.33%)

Class 1: 1 (16.67%)

$Gini_{left} = 1 - (0.83^2 + 0.17^2)$

$= 0.2778$

RIGHT CHILD

Labels: [1, 1, 1, 1]

Class 0: 0 (0%)

Class 1: 4 (100%)

$Gini_{right} = 1 - (0^2 + 1^2)$

$= 0.0000$

Weighted Gini = (6/10) × 0.2778 + (4/10) × 0.0000 = 0.1667

Gini Gain = $Gini_{parent}$ − Weighted Gini = 0.5000 − 0.1667 = **0.3333**

KEY TAKEAWAYS

- ✓ Gini measures impurity (0 = pure, 0.5 = max for binary).
- ✓ Weighted Gini is the impurity after the split.
- ✓ Gini Gain is the reduction in impurity.
- ✓ Choose the split with the highest Gini Gain.

REFERENCE

$$Gini = 1 - \sum_{i=1}^{K} p_i^2$$

$$Weighted\ Gini = \frac{N_L}{N} Gini_L + \frac{N_R}{N} Gini_R$$

$$Gini\ Gain = Gini_{parent} - Weighted\ Gini$$

4. Regression Trees

Predicting continuous values with trees

Regression Trees — How They Work

Same CART algorithm — targets are numbers instead of class labels

Split criterion: Minimize weighted MSE (variance reduction)

$$MSE(S) = (1/|S|) \times \Sigma (y_i - \bar{y}_s)^2$$

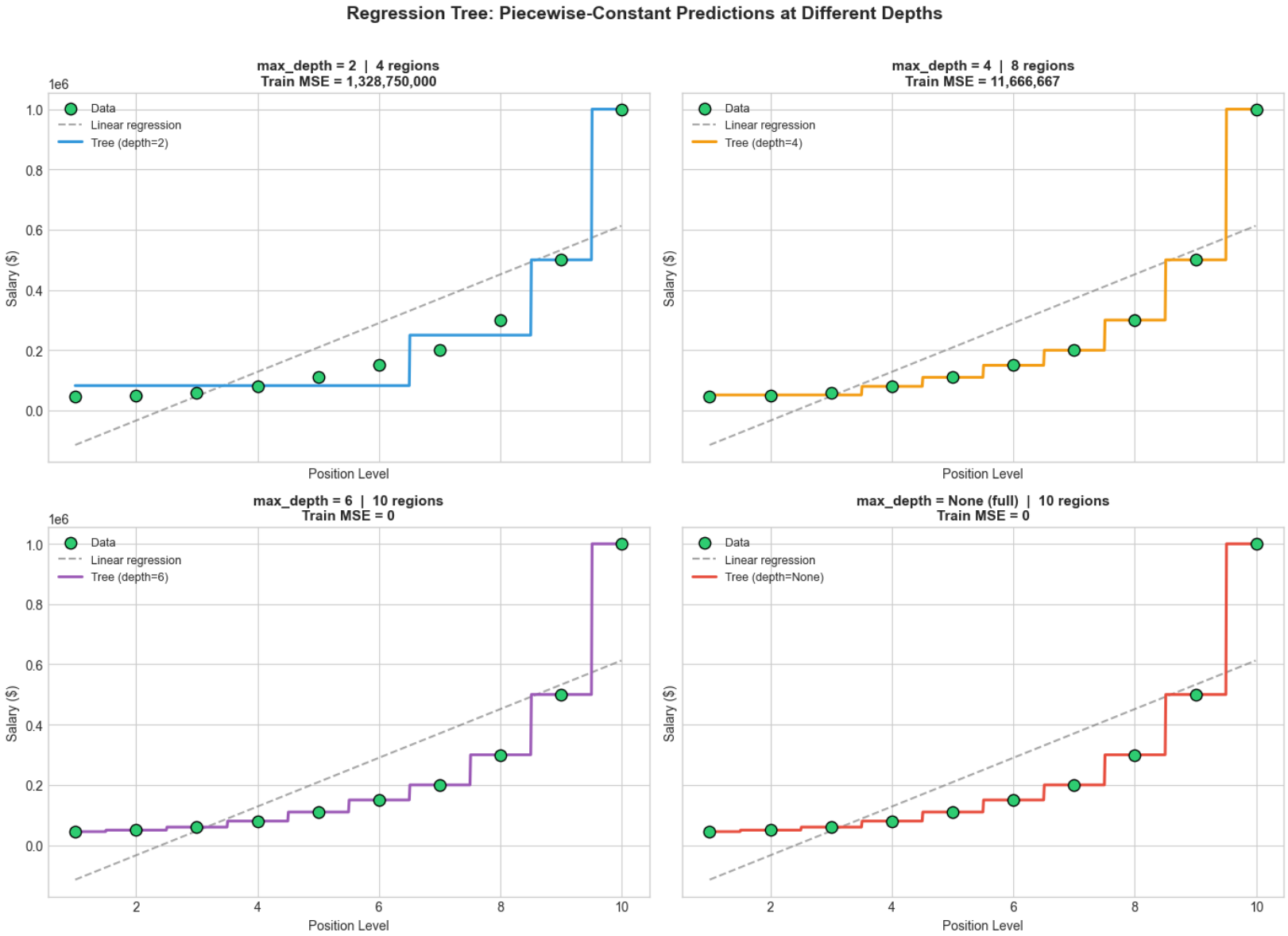
Leaf prediction:

- Each leaf predicts the MEAN of training targets in that region
- Result: piecewise-constant (step-function) predictions

⚠ Extrapolation failure:

- Trees cannot predict values outside the training range
- Input beyond max training value → returns last leaf mean
- Use neural nets or polynomial regression if extrapolation needed

Regression Tree: Depth Controls Granularity



5. Overfitting, Bias-Variance & Pruning

The most important concept in ML

The Bias-Variance Tradeoff

Every model faces this fundamental tension:

Too SIMPLE (shallow tree, max_depth=1):

- High Bias — misses real patterns in data
- Low Variance — stable but wrong
- → Underfitting

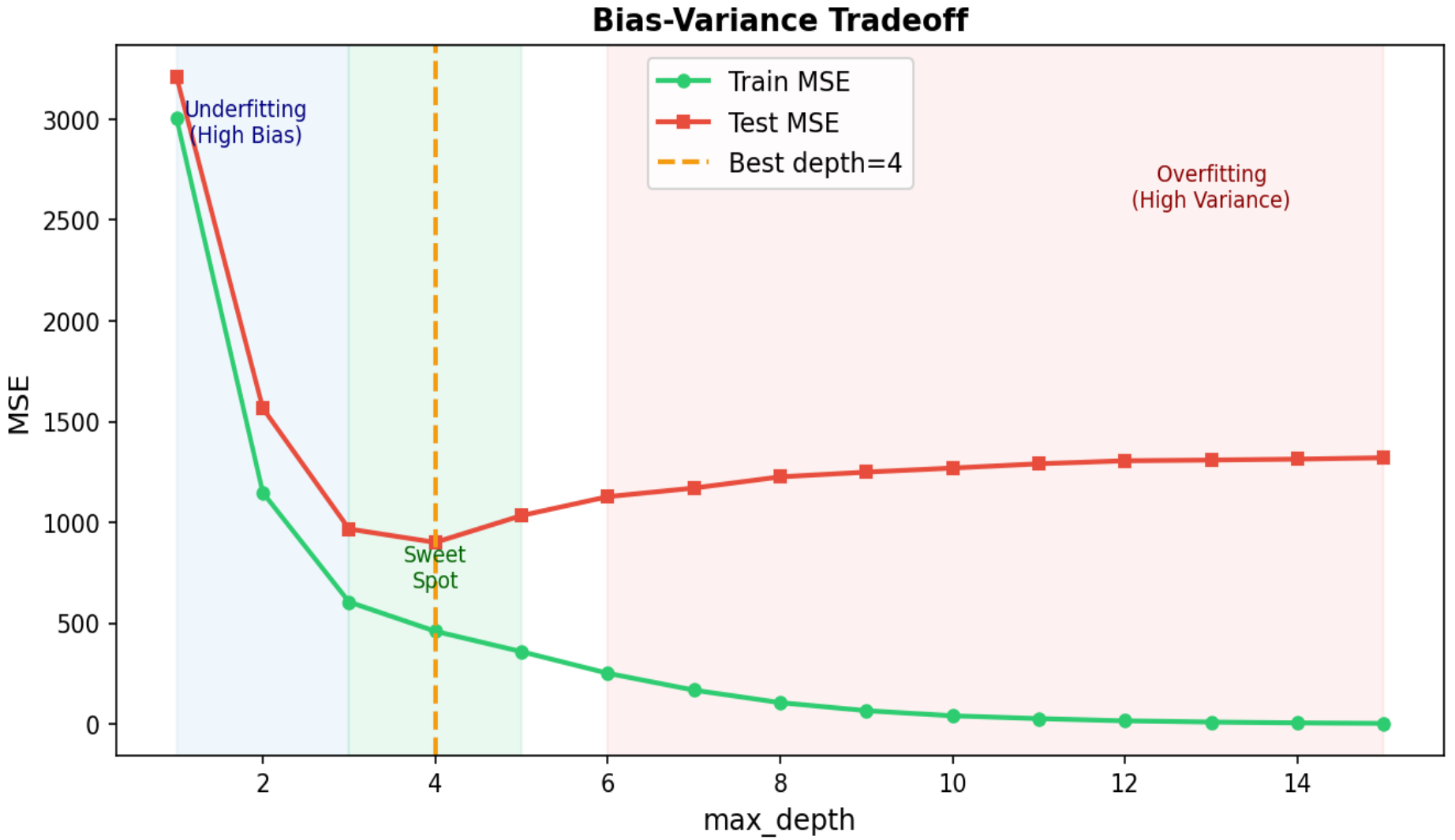
Just RIGHT:

- Low Bias + Low Variance → Good generalisation

Too COMPLEX (deep tree, no limits):

- Low Bias — memorises every training point
- High Variance — tiny data change = totally different tree
- → Overfitting (Train acc=1.0, Test acc much lower)

Bias-Variance Tradeoff — Visualised



Controlling Overfitting

Pre-pruning (stop early while growing):

max_depth — hard limit on tree depth (start: 5)

min_samples_split — min samples needed to split a node (default: 2)

min_samples_leaf — min samples required in any leaf (default: 1)

max_leaf_nodes — cap total number of leaves

Post-pruning (grow full tree, then simplify):

ccp_alpha — cost-complexity pruning (higher → simpler tree)

Cross-validation:

Use GridSearchCV or learning curves to find the sweet spot

- Or just use Random Forest — built-in variance reduction!

6. Random Forest

The Power of the Crowd

Random Forest Algorithm

RANDOM FOREST — HOW IT WORKS

FOR i = 1 to n_estimators:

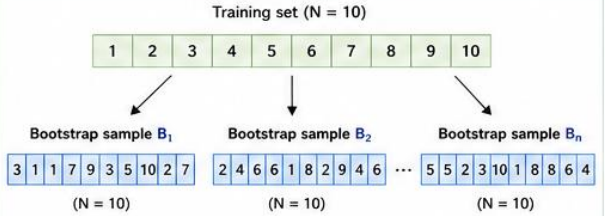
1. Draw bootstrap sample B_i (N points, with replacement)
   - Each tree sees a different sample of the training data.
2. Grow decision tree T_i on B_i:
   - At each split: randomly select $\sqrt{p}$ features
   - Pick best split among those $\sqrt{p}$ features
   - Grow until stopping criterion (max_depth, min_samples_leaf, ...)

Predict new point x:

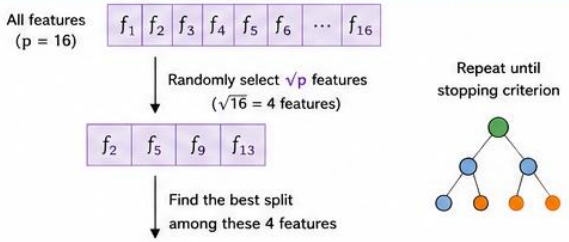
- Classification: $y = \text{majority\_vote}(T_1(x), T_2(x), ..., T_n(x))$
- Regression: $y = \text{mean}(T_1(x), T_2(x), ..., T_n(x))$

Key idea:
Randomness (bootstrap + random feature selection) makes trees diverse → reduces variance → improves generalization.

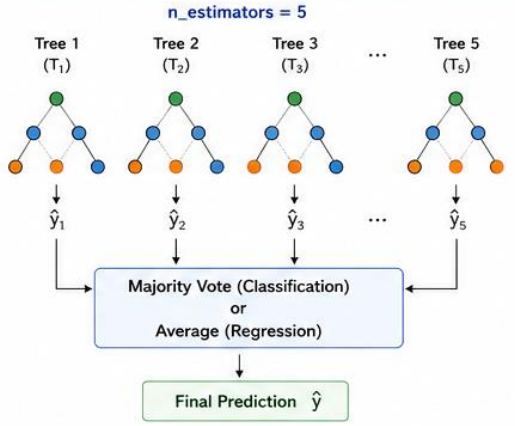
1. BOOTSTRAP SAMPLING (with replacement)



2. RANDOM FEATURE SELECTION AT EACH SPLIT



3. RANDOM FOREST PREDICTION



- Reduces overfitting (lower variance)
- Works well for high-dimensional data
- Robust to noise and outliers

4. CODE EXAMPLE (Scikit-learn)

```python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,      # number of trees
    max_depth=10,          # max depth of each tree
    oob_score=True,        # enable out-of-bag score (free estimate)
    n_jobs=-1,             # use all CPU cores
    random_state=42        # for reproducibility
)

rf.fit(X_train, y_train)

# After training:
# rf.predict(X_test)          -> make predictions
# rf.oob_score_               -> OOB score (if oob_score=True)
# rf.feature_importances_     -> feature importance
```

Tip: Set oob_score=True to get a built-in cross-validation score using out-of-bag samples (no extra CV needed).

5. KEY PARAMETERS

Parameter	Meaning	Typical Values / Notes
n_estimators	Number of trees	100, 200, 300, 500, ...
max_depth	Maximum depth of each tree	None, 5, 10, 20, ... (None = grow until pure)
min_samples_split	Min samples to split an internal node	2, 5, 10, ...
min_samples_leaf	Min samples in any leaf	1, 2, 4, ...
max_features	Features per split	"sqrt", "log2", 0.5, 0.7, 1.0 (sqrt is default for classification)
ccp_alpha	Cost complexity pruning strength	0.0 (default), 0.001, 0.01, ...
oob_score	Use OOB samples to estimate score	True / False
n_jobs	Number of CPU cores	-1 (all cores)
random_state	Random seed	Any integer (e.g., 42)

OOB (Out-of-Bag): For each tree, ~1/3 of samples are left out and used to evaluate that tree. Aggregated to get OOB score.

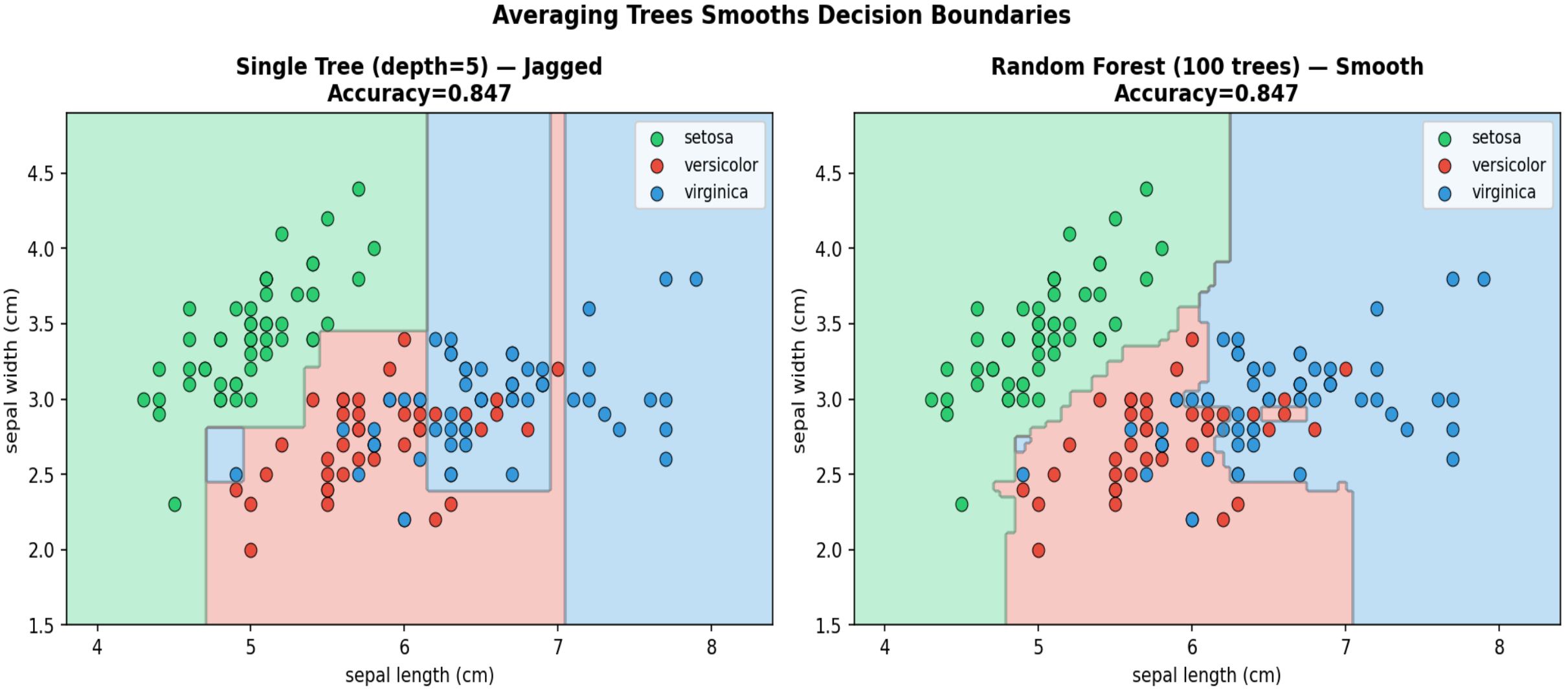
6. RULE OF THUMB (TUNING ORDER)

1. Start with **defaults** — already strong baseline.
2. Tune **max_depth** first (try: None, 5, 10, 20).
3. Then tune **min_samples_split** and **min_samples_leaf**.
4. Use **oob_score=True** for quick evaluation.
5. Use **GridSearchCV / RandomizedSearchCV** with **cv=5** for final tuning.

7. KEY TAKEAWAYS

- Bootstrap sampling + random feature selection = diverse trees.
- Diversity reduces variance and improves generalization.
- Use more trees (n_estimators) for better stability.
- Control tree complexity with max_depth and min_samples_*.
- OOB score gives a free, reliable estimate of performance.
- Random Forests are robust, scalable, and widely used!

Single Tree vs Random Forest — Decision Boundaries



Out-of-Bag (OOB) Score — Free Validation

Each tree only sees ~63% of data:

The remaining ~37% (OOB) were never used to train that tree

Evaluate each tree on its OOB set → aggregate = OOB score

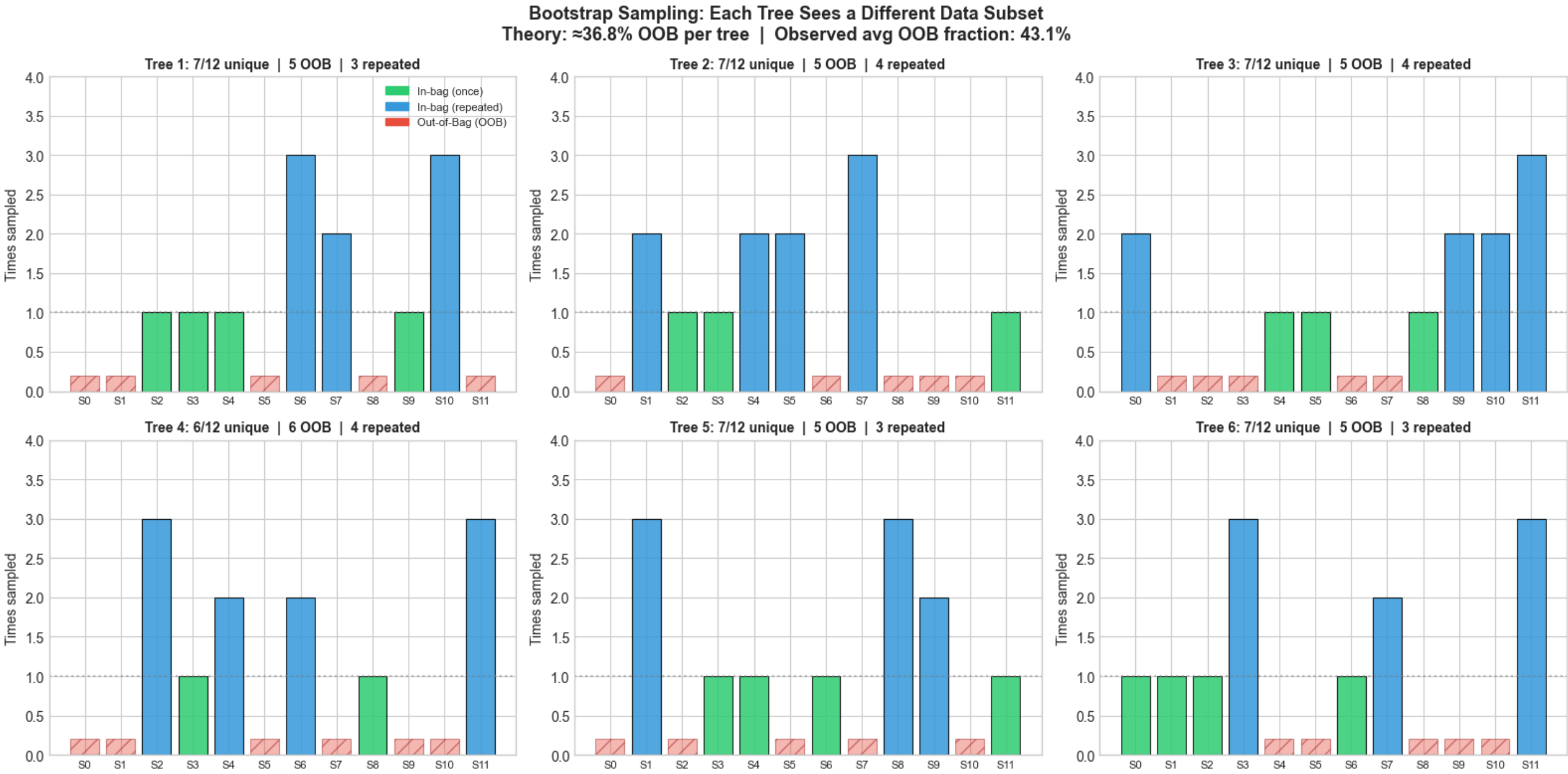
- OOB score ≈ cross-validation score (no need for a separate val set)
- OOB error decreases as n_estimators increases
- Stabilises after ~50–100 trees

- Adding more trees NEVER causes RF overfitting (unlike boosting)

Code:

```
rf = RandomForestClassifier(oob_score=True)
rf.fit(X, y)
print(rf.oob_score_)   # free accuracy estimate
```

Bootstrap Sampling — Each Tree Sees Different Data



Random Forest — Key Ingredients

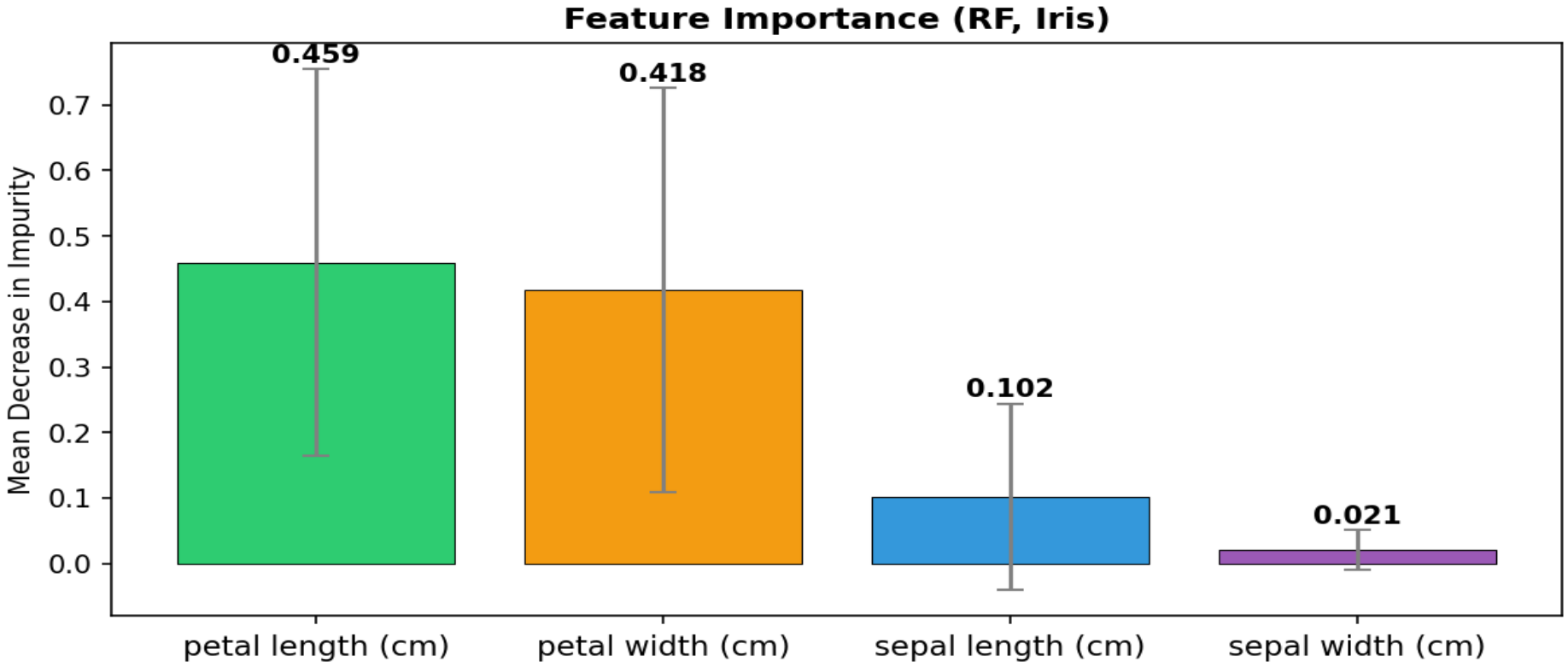
Problem: A single tree has HIGH VARIANCE
Small data change → completely different tree
Solution: Train many diverse trees, average their predictions

1. Bagging (Bootstrap Aggregating)
   - Each tree trains on a random bootstrap sample (~63% unique)
   - ~37% OOB samples = free validation set (no extra data needed)

2. Feature Randomness (the key innovation)
   - At each split: only $\sqrt{p}$ features considered (classification)
   - Forces different trees to rely on different features
   - De-correlates the trees → stronger variance reduction

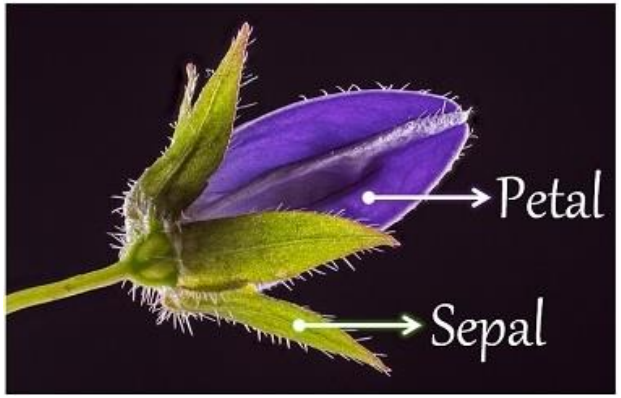
3. Aggregation
   - Classification: majority vote | Regression: mean prediction

Feature Importance — Which Variables Matter Most?



- MDI = Mean Decrease in Impurity: average Gini reduction this feature causes, weighted by samples
- Petal features dominate Iris — makes biological sense

⚠ MDI can be biased toward high-cardinality features. Use permutation importance for critical tasks.



7. Hyperparameter Tuning — Key Parameters

RandomForestClassifier — Key Parameters Explained

1. CODE EXAMPLE

```
RandomForestClassifier(
    n_estimators      = 100,     # number of trees
    max_depth         = None,    # None = grow until pure ⚠ set this!
    min_samples_split = 2,       # min samples to allow a split
    min_samples_leaf  = 1,       # min samples in any leaf
    max_features      = "sqrt",  # features per split (√p for classification)
    ccp_alpha         = 0.0,     # post-pruning strength
    oob_score         = False,   # set True for free OOB estimate
    n_jobs            = -1,      # -1 = use all CPU cores
    random_state      = 42,      # reproducibility
)
```

3. RULE OF THUMB (TUNING ORDER)

1. **Start with defaults**
   They are often already strong.
2. **Tune max_depth first**
   Try: 5, 10, 20, None
   Limits tree complexity.
3. **Then tune min_samples_split and min_samples_leaf**
   Try: 2, 5, 10 (split) and 1, 2, 4 (leaf)
   Helps control overfitting.
4. **Use GridSearchCV with cv=5**
   Find the best combination using cross-validation.

Why this order?

- ✓ max_depth has the biggest impact on complexity.
- ✓ min_samples_split and min_samples_leaf refine the tree size.
- ✓ GridSearchCV finds the optimal balance.

2. PARAMETERS EXPLAINED

Parameter	Type	Default	What it does	When to adjust
n_estimators	int	100	Number of trees in the forest.	Increase for better performance (more stable), but slower.
max_depth	int or None	None	Maximum depth of each tree. None = grow until leaves are pure.	Limit depth to prevent overfitting.
min_samples_split	int	2	Minimum samples required to split an internal node.	Increase to reduce overfitting (simpler trees).
min_samples_leaf	int	1	Minimum samples required in a leaf node.	Increase to reduce overfitting and get smoother predictions.
max_features	int, float or str	"sqrt"	Number of features to consider at each split. "sqrt" is common for classification.	Try "sqrt", "log2" or a fraction (e.g., 0.5) to control randomness.
ccp_alpha	float	0.0	Minimal Cost-Complexity Pruning parameter. 0.0 = no pruning.	Increase to prune trees and reduce overfitting.
oob_score	bool	False	Use out-of-bag samples to estimate generalization score.	Set True to get OOB score (works only with bootstrap=True).
n_jobs	int	None	Number of CPU cores to use. -1 uses all cores.	Set -1 to speed up training (on multi-core machines).
random_state	int	None	Controls randomness for reproducible results.	Set a number to make results reproducible.

4. COMMON VALUE GUIDELINES

Parameter	Typical Values to Try
n_estimators	100, 200, 300, 500
max_depth	5, 10, 20, **None**
min_samples_split	2, 5, 10
min_samples_leaf	1, 2, 4, 8
max_features	"sqrt", "log2", 0.5, 0.7, 1.0
ccp_alpha	0.0, 0.001, 0.01, 0.1
oob_score	False, True
n_jobs	-1
random_state	42 (or any fixed number)

5. BEST PRACTICES

- **Monitor overfitting**
  Compare train vs validation/OOB score.
- **Use OOB score**
  Set `oob_score=True` for a quick out-of-bag estimate.
- **Balance bias–variance**
  Shallower trees (small `max_depth`) reduce variance.
- **Use all cores**
  Set `n_jobs=-1` for faster training.
- **Fix random_state**
  Ensures reproducible results.

6. QUICK SUMMARY

More trees (n_estimators)	Limit depth (max_depth)	Increase min_samples_split / min_samples_leaf	max_features adds randomness	Tune smartly, validate properly, ship confidently!
→ more stability	→ control complexity	→ reduce overfitting	→ better generalization	

TIP: Use **Cross-Validation** (cv=5) and evaluate with **accuracy**, precision, recall, and F1-score.

Code: GridSearchCV

GridSearchCV in Scikit-learn (Example)

```python
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    "n_estimators":      [50, 100, 200],
    "max_depth":         [5, 10, 20, None],
    "min_samples_split": [2, 5, 10],
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1,
)
grid.fit(X_train, y_train)
```

```
Best params : {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 200}
Best CV acc : 0.9421
```

```
best_model = grid.best_estimator_
```

1) HYPERPARAMETER GRID

Parameter	Values Tried
n_estimators	50, 100, 200
max_depth	5, 10, 20, None
min_samples_split	2, 5, 10

SEARCH SPACE SIZE

3 (n_estimators) × 4 (max_depth) × 3 (min_samples_split)
= **36** combinations

With 5-fold Cross Validation:
36 combinations × 5 folds = **180** training runs

WHAT EACH HYPERPARAMETER DOES

n_estimators	Number of trees in the forest. More trees → higher stability, but more computation.
max_depth	Maximum depth of each tree. Limits how deep a tree can grow. None = unlimited depth.
min_samples_split	Minimum number of samples required to split an internal node. Larger values → less overfitting.

GRID SUMMARY

- Total parameter combinations: 36
- Cross-validation folds: 5
- Total model evaluations: 180

Goal: Find the combination that gives the highest average cross-validated accuracy.

2) 5-FOLD CROSS VALIDATION (cv=5)

1. Data is split into 5 folds.
2. For each fold:
   - Train on 4 folds
   - Validate on the 1 remaining fold
3. Repeat for all 5 folds.
4. Average the accuracy.
5. Do this for every parameter combination.

Dataset →

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Score
Iteration 1	Val	Train	Train	Train	Train	Score 1
Iteration 2	Train	Val	Train	Train	Train	Score 2
Iteration 3	Train	Train	Val	Train	Train	Score 3
Iteration 4	Train	Train	Train	Val	Train	Score 4
Iteration 5	Train	Train	Train	Train	Val	Score 5

Average Accuracy = (Score 1 + Score 2 + ... + Score 5) / 5

3) RESULTS

Example Output

```
Best params : {'max_depth': 10,
               'min_samples_split': 2,
               'n_estimators': 200}
Best CV acc : 0.9421
```

Best hyperparameters found by GridSearchCV

Highest average accuracy across 5 folds

```
best_model = grid.best_estimator_
```

Best model retrained on the full training data with the best parameters

KEY TAKEAWAYS

- ✅ GridSearchCV tries every combination of hyperparameters.
- ✅ It uses **cross-validation** to get a reliable estimate of performance.
- ✅ It returns the best hyperparameters and the best trained model.
- ✅ Helps build models that generalize better to unseen data.

Code: Full Model Evaluation (for next week)

Classification Evaluation with Scikit-learn

1. CODE WALKTHROUGH

```python
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

y_pred = rf.predict(X_test)

# Confusion matrix
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred, display_labels=class_names).plot()

# Per-class metrics
print(classification_report(y_test, y_pred, target_names=class_names))

#              precision    recall  f1-score   support
# setosa           1.00      1.00      1.00        15
# versicolor       1.00      0.93      0.97        15
# virginica        0.94      1.00      0.97        15

# ⚠ Accuracy alone hides per-class failures!
# Always check precision, recall, and F1.
```

2. CONFUSION MATRIX EXPLAINED

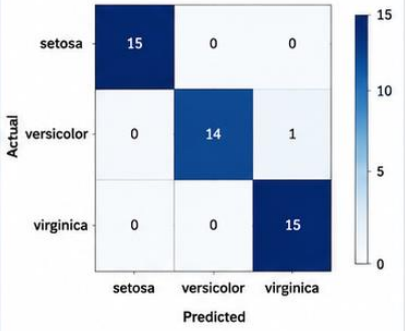
Rows = Actual (True) labels
Columns = Predicted labels

Actual \ Predicted	Setosa	Versicolor	Virginica
Setosa	TP (True Positive)	FP (False Positive)	FP (False Positive)
Versicolor	FN (False Negative)	TP (True Positive)	FP (False Positive)
Virginica	FN (False Negative)	FN (False Negative)	TP (True Positive)

TP: Correctly predicted as the class
FP: Incorrectly predicted as the class
FN: Actual class but predicted as something else
TN: Correctly predicted as not the class (not shown in 3-class matrix)

3. EXAMPLE CONFUSION MATRIX (Heatmap)

Iris Dataset (Example)
Classes: setosa, versicolor, virginica



Diagonal values = correct predictions
Off-diagonal values = misclassifications

4. CLASSIFICATION REPORT OUTPUT

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	15
versicolor	1.00	0.93	0.97	15
virginica	0.94	1.00	0.97	15
accuracy	-	-	0.97	45
macro avg	0.98	0.98	0.98	45
weighted avg	0.98	0.97	0.97	45

Precision: Of all predicted as class, how many are correct?
Recall: Of all actual class samples, how many did we find?
F1-score: Harmonic mean of **Precision** and **Recall**
Support: Number of actual occurrences of the class

5. METRICS EXPLAINED

Precision
Of all samples predicted as this class, how many are actually this class?

$$Precision = \frac{TP}{TP + FP}$$

Recall (Sensitivity)
Of all actual samples of this class, how many did we correctly predict?

$$Recall = \frac{TP}{TP + FN}$$

F1-score
Harmonic mean of Precision and Recall. Balances both metrics.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Why not just Accuracy?
- Accuracy can be misleading on imbalanced data.
- Per-class metrics reveal how the model behaves for each class.

6. BEST PRACTICES

- Always inspect the confusion matrix to see where errors occur.
- Check precision, recall, and F1-score for each class, not just accuracy.
- Focus on recall for rare classes (don't miss positive cases).
- Focus on precision to reduce false alarms.
- Report macro avg and weighted avg for overall performance.

QUICK TAKEAWAY
Use Confusion Matrix + Classification Report together for a complete evaluation.

Summary Decision Trees & Random Forests

Concept	Core Idea
CART Algorithm	Greedy splits minimising Gini/MSE at each node
Axis-Aligned Splits	Boundaries are always perpendicular to feature axes
Gini ≈ Entropy	Shapes nearly identical; Gini faster (no log)
Regression Tree	Predicts leaf mean → step-function output
Bias-Variance	Shallow = underfitting | Deep = overfitting
Bootstrap Sampling	≈63% unique per tree | ≈37% OOB = free validation
Feature Randomness	Only √p features per split → diverse trees
Random Forest	Average diverse trees → lower variance, no overfit
Feature Importance	Mean Gini reduction across all trees (MDI)

8. Practice Exercises

Conceptual:

Q1. Node has 8 class A, 2 class B. Compute Gini impurity. [Ans: 0.32]

Q2. Why does max_depth=None almost always overfit?

Q3. Why do more RF trees NOT cause overfitting?

Q4. Why can't trees extrapolate? Give a concrete example.

Coding:

E1. load_wine() → DecisionTreeClassifier(max_depth=4) → plot_tree() → export_text()

E2. RandomForestClassifier(oob_score=True) on wine → compare OOB vs 5-fold CV

E3. Plot train vs test accuracy for max_depth=1..20 → find sweet spot

E4. (Challenge) Telco churn CSV → preprocess → RF → confusion matrix → top-5 features

Questions?

Decision Trees & Random Forests — Lecture 5 | Chen Hajaj